Sports Analytics: A Dynamic Machine Learning Framework for Forecasting Test Cricket Outcome

A report submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Science

in

Electrical Engineering and Computer Science

by

VIPUL SHARMA

22369

DEPARTMENT OF ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

INDIAN INSTITUTE OF SCIENCE EDUCATION AND RESEARCH BHOPAL

Bhopal - 462 066

April, 2026

CERTIFICATE

This is to certify that **Vipul Sharma** , BS (Electrical Engineering and Computer Science) has worked on the project entitled **'Sports Analytics: A Dynamic Machine Learning Framework for Forecasting Test Cricket Outcomes'** under my supervision and guidance. The content of this report is original and has not been submitted elsewhere for the award of any academic or professional degree.

April, 2026
IISER Bhopal

Dr. Kundan Kandhway
Assistant Professor,
Department of Electrical Engineering
and Computer Science
IISER Bhopal

ACADEMIC INTEGRITY AND COPYRIGHT DISCLAIMER

I hereby declare that this thesis is my own work and, to the best of my knowledge, it contains no materials previously published or written by any other person, or substantial proportions of material which have been accepted for the award of any other degree or diploma at IISER Bhopal or any other educational institution, except where due acknowledgement is made in the thesis.

I certify that all copyrighted material incorporated into this thesis is in compliance with the Indian Copyright (Amendment) Act, 2012 and that I have received written permission from the copyright owners for my use of their work, which is beyond the scope of the law. I agree to indemnify and save harmless IISER Bhopal from any and all claims that may be asserted or that may arise from any copyright violation.

April, 2026
IISER Bhopal

Vipul Sharma

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Dr. Kundan Kandhway, for his invaluable guidance, constant support, and insightful discussions throughout the course of this work. His perspective and constructive feedback have been instrumental in shaping both the direction and the quality of this thesis.

I am also grateful to the faculty members of the Department of Electrical Engineering and Computer Science at IISER Bhopal for providing a strong academic foundation and a stimulating research environment. The coursework and interactions over the years have played a crucial role in developing the skills necessary for this work.

I would like to acknowledge my peers and collaborators for their valuable discussions and support. Their inputs and shared experiences have significantly contributed to improving both the technical and conceptual aspects of this project.

Finally, I am deeply grateful to my family for their unwavering encouragement and support. Their trust and patience have been a constant source of motivation throughout my academic journey

Vipul Sharma

ABSTRACT

Predicting the outcome of a Test cricket match presents a uniquely complex challenge in sports analytics due to its five-day duration, deteriorating pitch conditions, and the highly strategic nature of the 'Draw'. Traditional statistical models often fail to capture the non-linear shifts in momentum that define the format. This study introduces a real-time, in-play machine learning forecasting framework to predict Test match outcomes (Win, Draw, Loss) using ball-by-ball data from 899 Men's Test matches played between 2001 and 2026, transformed into discrete interval states.

Building upon the statistical phase-transition framework of Akhtar and Scarf (2012), this project engineers 21 dynamic covariates, prominently featuring non-linear partnership-based wicket resources, the cumulative match lead, and Relative Required Run Rate (RRR). Four distinct architectures—Multinomial Logistic Regression, Random Forest, XGBoost, and a Multi-Layer Perceptron (MLP)—were implemented and rigorously tuned using the Optuna optimization framework.

Timeline analysis across 15-over intervals demonstrates that tree-based ensembles significantly outperform standard linear baselines, Random Forest emerging as the peak performer in the late-match phase at 81 percent with XGBoost and Logistic Regression converging near 75 percent in the final 50 overs Feature importance extraction—utilizing Gain, Gini impurity, and permutation metrics—empirically validates that while historical team strength dictates early predictions, the cumulative lead and remaining wicket resources become the absolute determinants on the final day. The study concludes by generating dynamic probability timelines, proving that advanced ensemble methods can successfully track real-time match momentum and stalemate trajectories.

Table of Contents

Certificate **i**

Academic Integrity and Copyright Disclaimer **ii**

Acknowledgement **iii**

Abstract **iv**

Table of Contents **v**

List of Figures **viii**

List of Tables **viii**

1 Introduction **1**
1.1 Background and Context 1
1.2 Problem Statement 1
1.3 Research Objectives 2
1.4 Significance of the Study 2

2 Literature Review **3**
2.1 Introduction 3
2.2 Statistical Frameworks: Akhtar and Scarf (2012) 3
2.2.1 Machine Learning and Visualization: Agrawal 3
2.3 Research Gap and Objective 4

3 Methodology **5**
3.1 Data Preprocessing and State Representation 5
3.2 The Reference Team Framework 5
3.2.1 Outcome Labeling 5
3.3 Mathematical Formulation of Features 5
3.3.1 Static and Contextual Features 6
3.3.2 Dynamic In-Play Features 6
3.4 Preprocessing Pipeline 8

3.4.1 Encoding the Target Variable 8
3.4.2 Data Split and Leakage Prevention 8
3.4.3 Removal of Categorical Team Identifiers 8
3.5 Model Architectures 9
3.5.1 Logistic Regression 9
3.5.2 Random Forest Classifier 9
3.5.3 XGBoost (eXtreme Gradient Boosting) 10
3.5.4 Multi-Layer Perceptron (Neural Network) 10
3.5.5 Hyperparameter Optimization (Optuna) 10

4 Results and Discussion **12**
4.1 Exploratory Data Analysis (EDA) 12
4.1.1 Relative Strength (Win Percentage Difference) 12
4.1.2 Home Factor by Match Result 13
4.1.3 Toss Win by Match Result 13
4.1.4 First Innings Total 14
4.1.5 Match Lead in Innings 4 14
4.1.6 Wickets Remaining in Innings 4 15
4.1.7 Overs Remaining in Innings 4 16
4.2 Model Performance and Evaluation 16
4.2.1 Baseline Classifier 16
4.2.2 Combined Model Accuracy Analysis 17
4.2.3 XGBoost Performance Metrics 19
4.2.4 Random Forest Performance Metrics 20
4.2.5 Logistic Regression Performance Metrics 21
4.2.6 Neural Network Performance Metrics 23
4.2.7 Cross-Model Feature Importance Synthesis 24
4.3 In-Play Probability Timeline Analysis 26
4.3.1 Scenario 1: Collapse (Match ID: 1876) 26
4.3.2 Scenario 2: Recovery (Match ID: 1636) 27
4.3.3 Scenario 3: Draw (Match ID: 1917) 28
4.3.4 Summary 28

5 Conclusion and Future Work **29**
5.1 Conclusion 29
5.2 Future Work 30
5.2.1 Innings-Specific Architectures 30
5.2.2 Advanced Deep Learning (DL) Techniques 30

Appendices **31**

A Data Dictionary **32**

B Hyperparameter Configurations **34**

List of Publications **36**

Bibliography **37**

List of Figures

4.1 Relative Strength: Win Percentage Difference by Match Result. 13
4.2 Home Factor: Influence of Venue on Match Result. 13
4.3 Toss Win: Match Result distribution relative to the toss outcome. 14
4.4 First Innings Total: Scoring trends by match result. 14
4.5 Match Lead: Cumulative run difference in the final innings. 15
4.6 Wicket Resources: Survival capacity in the final innings. 15
4.7 Overs Remaining: Time consumption trends. 16
4.8 Prediction accuracy of all four models across match progression. The dashed red line denotes the majority-class baseline (61.11%). Ensemble models sustain performance above the baseline, with accuracy rising steadily as the match approaches conclusion. 17
4.9 Confusion Matrix XGBoost 19
4.10 Feature Importance XGBoost 20
4.11 Confusion Matrix Random Forest 20
4.12 Feature Importance Random Forest 21
4.13 Confusion Matrix Logistic Regression 22
4.14 Feature Importance Logistic Regression 22
4.15 Confusion Matrix Neural Network 23
4.16 Feature Importance Neural Network 24
4.17 Scenario: Collapse (Match ID: 1876). The reference team builds a strong winning position through the mid-match phase before a sudden batting collapse inverts the probabilities in the final overs. 26
4.18 Scenario: Recovery (Match ID: 1636). The reference team begins from a losing position but mounts a sustained recovery, with the model progressively reassigning probability mass toward a Win as the lead and wicket resources improve. 27
4.19 Scenario: Draw (Match ID: 1917). The model initially favours a Win for the reference team but progressively transfers probability mass to Draw as diminishing overs and wicket resources signal an inconclusive finish. . . . 28

List of Tables

3.1 Partnership Resource Weights (%) 7

4.1 Feature importance rankings across all four models (lower = more important). Rankings are directionally comparable only; scores are not numerically equivalent across models. 25

A.1 Detailed Data Dictionary of Match State Features 33

B.1 XGBoost Optuna Search Space and Optimal Parameters 34
B.2 Random Forest Optuna Search Space and Optimal Parameters 35
B.3 MLP Optuna Search Space and Optimal Parameters 35

Chapter 1

Introduction

1.1 Background and Context

Cricket is one of the most widely followed sports globally, possessing a rich statistical history that lends itself naturally to quantitative analysis. However, the application of predictive modeling in cricket is largely bifurcated by format. Limited-overs cricket, such as One Day Internationals (ODIs) and Twenty20 (T20) matches, operates under strict ball constraints. In these formats, the objective is linear and universally understood: maximize run-scoring before resources (wickets or balls) run out. Consequently, predictive models for limited-overs cricket have reached a high level of maturity.

Test cricket, conversely, presents a fundamentally different and significantly more complex predictive environment. Spanning up to five days and 450 overs, a Test match is not merely a contest of run accumulation but a strategic battle of attrition. Pitch deterioration, weather interventions, fatigue, and the lack of an absolute ball constraint create non-linear shifts in momentum. Most importantly, Test cricket features a highly probable third outcome: the 'Draw'. Unlike a tie, a draw occurs when time runs out before a decisive result is reached, often resulting from deliberate defensive play rather than an inability to score.

This strategic depth means that traditional limited-overs models—which treat wickets purely as scoring resources—fail when applied to Test cricket, where wickets often represent time-consuming survival capital.

1.2 Problem Statement

The objective of this research is to develop a robust, real-time machine learning framework capable of predicting the outcome (Win, Draw, Loss) of a Test cricket match at any given state of play.

Predicting Test match outcomes requires solving two primary challenges:

1. **The Phase Transition Problem:** The factors that dictate a match outcome change drastically as the match progresses. Pre-match priors (like historical team strength and home-ground advantage) heavily influence the opening days but fade

into insignificance by the final innings, replaced entirely by dynamic in-play covariates (like the lead and remaining wicket resources).

2. **The Non-Linearity of the Draw:** Predicting a draw requires the model to understand the interaction between remaining batting resources and remaining overs, capturing the point at which a chasing team abandons the pursuit of a win to block for a stalemate.

Existing statistical models, such as Multinomial Logistic Regression, handle the phase transition well but struggle with the non-linear thresholds required to accurately predict draws. This study seeks to address this gap by implementing advanced tree-based ensemble methods and neural networks.

1.3 Research Objectives

To address the problem statement, this study establishes the following core objectives:

- To engineer a robust state-space representation of a Test match using a standardized "Reference Team" framework, enabling the calculation of dynamic features such as non-linear wicket resource depletion.
- To implement and tune four distinct machine learning architectures (Logistic Regression, Random Forest, XGBoost, and a Multi-Layer Perceptron) using advanced hyperparameter optimization techniques (Optuna).
- To comparatively evaluate these models across the 450-over duration of a match, determining the point at which in-play covariates statistically overpower pre-match priors.
- To extract feature importance metrics, analyzing the shifting weight of variables like the 'Lead' and 'Toss Win' to empirically validate domain knowledge.

1.4 Significance of the Study

While sports forecasting is often associated with betting markets, accurate in-play modeling has significant applications for broadcast media, team strategy, and fan engagement. By generating dynamic probability timelines, broadcasters can quantify the exact impact of a single wicket or a sustained partnership in real-time. Furthermore, establishing that machine learning ensembles can inherently capture the complex, non-linear nature of the 'Draw' provides a methodological step forward from traditional linear statistical models in sports analytics.

Chapter 2

Literature Review

2.1 Introduction

Predictive modeling in Test cricket presents a uniquely complex environment compared to limited-overs formats. Spanning five days, matches are governed by a "phase transition" where early pre-match priors (team strength, venue) are gradually replaced by dynamic in-play covariates (run lead, remaining wickets). This chapter reviews the foundational statistical frameworks and modern machine learning approaches that directly inform our methodology.

2.2 Statistical Frameworks: Akhtar and Scarf (2012)

The primary theoretical foundation for this study's feature engineering derives from Akhtar and Scarf (2012) [1] in their paper *"Forecasting Test match cricket outcomes in play"*. Their research introduced critical paradigms for Test match prediction:

- **The Phase Transition and Lead Dominance:** They empirically demonstrated that pre-match variables become statistically negligible as the match progresses. The cumulative run advantage (`Match_Lead`) eventually absorbs all other signals, becoming the dominant predictor in the final innings.

- **Non-Linear Wicket Resources:** Rather than treating wickets as linearly depleting assets, they proved that wickets possess non-linear, partnership-specific values. By weighting early partnerships heavier than tail-end wickets, they created a highly stable in-play metric for "survival capacity." This exact resource methodology was adopted in our feature pipeline.

2.2.1 Machine Learning and Visualization: Agrawal

While Akhtar and Scarf provided an optimal statistical definition of the match state, their reliance on Multinomial Logistic Regression inherently assumed linear relationships. To capture the complex, non-linear realities of a match—such as sudden batting collapses or rapid scoring shifts—our architecture and evaluation methods are heavily influenced by the work of Agrawal [2].

In the study on One Day International (ODI) cricket, Agrawal demonstrated how various supervised machine learning models perform when predicting the winner of an ongoing match. The evaluation of models like Random Forest and Neural Networks highlighted the strength of modern algorithms in adapting to dynamic, in-play features. Furthermore, our methodology adopted two key components from this approach to data visualization and evaluation:

- **Feature Importance Tracking:** Agrawal utilized explainable AI techniques (SHAP scores) to interpret how the importance of specific features changes state-by-state as the match progresses. Inspired by this, we utilize native model metrics (such as Gini Impurity and Information Gain) to ensure model interpretability and empirically validate cricket domain knowledge across different match phases.
- **Dynamic State-by-State Visualizations:** Rather than relying on static, end-of-match metrics, the study visualized prediction accuracies and feature importance on an over-by-over basis. This technique was adopted in our study to generate real-time probability tracking and visualize dynamic match scenarios (Collapse, Recovery, Draw) presented in our results.

2.3 Research Gap and Objective

The existing literature often isolates rigorous statistical feature engineering from advanced machine learning. Akhtar and Scarf offer deep domain specificity but rely on rigid linear models, while modern ML applications often target the simpler state space of T20 cricket.

The objective of this project is to bridge this gap. By combining the Test-specific feature definitions of Akhtar and Scarf with the non-linear predictive architectures and dynamic visualizations demonstrated by Agrawal, this study proposes a state-of-the-art hybrid framework for real-time Test match forecasting.

Chapter 3

Methodology

3.1 Data Preprocessing and State Representation

The dataset derived from Cricsheet's [3] ball-by-ball JSON records required a transformation into a state-space representation. Unlike limited-overs cricket, the temporal dynamics of Test cricket span up to 450 overs . To capture strategic shifts without losing granular information, we define the match state S_t at discrete intervals t of 5 overs (30 balls).

Each record in the processed CSV corresponds to one such interval. This resolution allows the model to observe changes in scoring rates and wicket resource depletion at a frequency that matches typical session-based strategy adjustments in professional play.

3.2 The Reference Team Framework

To maintain mathematical consistency and eliminate team-specific bias, this study employs a **Reference Team** (T_{ref}) framework.

- T_{ref} is defined as the team that bats first in the match.
- All features are calculated relative to T_{ref}.
- The target outcome y is centered on T_{ref}.

3.2.1 Outcome Labeling

The classification objective is to predict the final match result $y \in \{-1, 0, 1\}$, formulated as:

$$y = \begin{cases} 1 & \text{if } T_{ref} \text{ wins the match} \\ 0 & \text{if the match results in a Draw} \\ -1 & \text{if } T_{ref} \text{ loss the match} \end{cases} \tag{3.1}$$

3.3 Mathematical Formulation of Features

Based on the feature space present in the processed dataset, we categorized our 21 features into static identifiers, pre-match predictors, and dynamic state-based variables.

3.3.1 Static and Contextual Features

These variables are computed at the start of the match ($t = 0$) or remain constant across intervals:

1. **Home Factor (H)**: A binary indicator capturing the local advantage.

$$H = \begin{cases} 1 & \text{if } T_{ref} \text{ has home advantage} \\ 0 & \text{otherwise} \end{cases} \tag{3.2}$$

2. **Toss Win (TW)**: A binary indicator representing the strategic advantage of winning the toss.

$$TW = \begin{cases} 1 & \text{if } T_{ref} \text{ won the toss} \\ 0 & \text{otherwise} \end{cases} \tag{3.3}$$

3. **Win Percentage Difference (ΔWP):** This feature quantifies the historical disparity in team strength prior to the match. Let WP_{ref} and WP_{opp} represent the historical win rates of the Reference Team and the Opponent, respectively. These individual win rates are calculated as:

   - $WP_{ref} = \frac{\text{Total Past Wins of Reference Team}}{\text{Total Past Matches of Reference Team}}$
   - $WP_{opp} = \frac{\text{Total Past Wins of Opponent}}{\text{Total Past Matches of Opponent}}$

   The final feature is then computed as the difference between these two values, representing the relative historical advantage:

$$\Delta WP = WP_{ref} - WP_{opp} \tag{3.4}$$

4. **Ground Effect (G)**: The venue-specific draw probability.

$$G = \frac{\text{Historical Draws at Venue}}{\text{Total Matches at Venue}} \tag{3.5}$$

3.3.2 Dynamic In-Play Features

Dynamic features capture the evolving match state X_t.

1. Lead and Run Progression

- **Interval Runs**: The number of runs scored by the batting team in the current 5-over interval.

- **Match Lead (L_t)**: The total run advantage or deficit of T_{ref} across all innings.

$$L_t = \sum \text{Runs}(T_{ref}, t) - \sum \text{Runs}(T_{opp}, t) \tag{3.6}$$

2. Wicket Resource Weighting (WR_t)

Wickets are treated as non-linear assets following Akhtar and Scarf (2012). Let k be the wickets lost. The remaining resource $WR(k)$ is the sum of partnership values ω_i.

$$WR(k) = \sum_{i=k+1}^{10} \omega_i \tag{3.7}$$

The weights ω_i (% values) for each wicket $i \in \{1 \dots 10\}$ are:

Table 3.1: Partnership Resource Weights (%)

Partnership	1st	2nd	3rd	4th	5th	6th	7th	8th	9th	10th
Value (ω_i)	12.24	12.13	11.51	10.82	10.29	9.48	8.88	8.22	7.69	9.24

3. Temporal and Strategic Indicators

- **Overs Remaining**: Counted down from 450 overs (max length).
- **Innings Num**: The current innings (1 to 4).
- **Batting Team Code**: 1 if T_{ref} is batting, 0 if the opponent is batting.
- **Follow-On :** A dynamic binary flag set to 1 if the follow-on rule is enforced, and 0 otherwise. By the strict definition of our dataset, the "Reference Team" is always the team batting first. Therefore, a follow-on scenario uniquely means the Opponent (batting in the 2nd innings) is forced to bat consecutively in the 3rd innings. Consequently, a value of 1 exclusively implies the follow-on was enforced *by* the Reference Team. To prevent target leakage, this feature is strictly set to 0 during the first two innings and only activates if the match reaches a follow-on state in the third innings.
- **Relative Required Run Rate (`Relative_RRR`):** This feature replaces traditional, backward-looking run rates with a forward-looking pressure metric. It is exclusively active during the target-chasing phase (the 4th innings) and is calculated as:

  $$Relative_RRR = \frac{\texttt{Match_Lead}}{\texttt{Overs_Remaining}} \tag{3.8}$$

  This metric is intentionally directional: a positive value indicates a mathematically favorable state for the Reference Team, while a negative value represents an unfavorable state. During the 1st, 2nd, and 3rd innings, this feature defaults to 0 to prevent noise.

- **Defensive Stance Indicator (`Defensive_Stance`):** A dynamic binary feature that is set to 1 when the batting team is playing defensively, and 0 otherwise.

  Calculation: This stance is detected algorithmically by tracking the team's scoring pace alongside their remaining survival capacity. The feature activates only if the batting team's run rate drops below 1.5 runs per over for three consecutive 5-over intervals, provided they still retain a high percentage of their wicket resources.

  Significance: This feature is crucial for predicting the 'Draw' outcome. It helps the machine learning model explicitly identify when a chasing team has abandoned the pursuit of a win and shifted to a deliberate blocking strategy. By checking that the team still has high wicket resources intact, the model can successfully distinguish between a team making a strategic choice to force a stalemate and a team that is simply scoring slowly because they are suffering a batting collapse.

3.4 Preprocessing Pipeline

Before feeding the engineered features into the machine learning architectures, a strict data preprocessing pipeline was implemented to ensure generalizability and prevent data leakage.

3.4.1 Encoding the Target Variable

The target variable representing the match outcome (`Result`) originally contained categorical string classes (Win, Loss, Draw). To make this compatible with the underlying mathematical models, it was transformed into a discrete numeric array (0, 1, 2) using Scikit-Learn's `LabelEncoder`.

3.4.2 Data Split and Leakage Prevention

A critical challenge in in-play sports modeling is data leakage. Since a single match generates up to 90 distinct interval states, a standard random train-test split would scatter states from the exact same match across both the training and testing datasets, allowing the model to artificially "peek" at the match trajectory.

3.4.3 Removal of Categorical Team Identifiers

During the final feature selection phase, explicit team identifiers (such as the Reference Team Name and Opponent Team Name) were deliberately excluded from the training dataset.

The objective of this framework is to evaluate the predictive power of dynamic, in-play match states (such as the cumulative lead, wicket resources) rather than relying on static historical biases. By dropping these categorical identifiers, we force the machine learning architectures to learn a universally generalizable representation

of a Test match. This strictly prevents the models from memorizing specific team names to predict outcomes, ensuring that a 150-run lead is evaluated purely on its mathematical merit, regardless of which countries are playing.

3.5 Model Architectures

To rigorously evaluate the predictive power of in-play match covariates, we implemented and compared four distinct machine learning architectures. These range from interpretable statistical baselines to highly complex non-linear ensembles, allowing us to determine the optimal approach for tabular Test cricket data.

3.5.1 Logistic Regression

Following the foundational methodology of Akhtar and Scarf (2012), Multinomial Logistic Regression was employed as our primary statistical baseline. It models the probability of each match outcome (Win, Draw, Loss) as a linear combination of the input features X_t. The probability of class j given the state vector is defined via the softmax function:

$$P(y = j \mid X_t) = \frac{e^{\beta_j^T X_t}}{\sum_{k=1}^{3} e^{\beta_k^T X_t}}$$

This model was selected for its high interpretability. The standardized coefficients (β) provide a direct, linear measure of feature importance, allowing us to cleanly observe how the weight of pre-match priors fades as in-play features like the `Lead` grow in magnitude.

3.5.2 Random Forest Classifier

Random Forest is an ensemble learning method that constructs a multitude of decision trees during training [4]. It utilizes bootstrap aggregating (bagging) and random feature selection to reduce the variance of individual decision trees. The final prediction is determined by a majority vote across all trees:

$$\hat{y} = \text{mode}\{h_1(X_t), h_2(X_t), \ldots, h_B(X_t)\}$$

where h_b represents an individual tree. This architecture was chosen because it inherently captures non-linear interactions between variables (such as `Wicket_Resources` and `Overs_Remaining`) while remaining highly robust against overfitting during the high-variance middle sessions of a match.

3.5.3 XGBoost (eXtreme Gradient Boosting)

Unlike Random Forest, which builds trees independently in parallel, XGBoost [5])builds trees sequentially. Each new tree attempts to correct the residual errors of the previous ensemble by minimizing a regularized objective function:

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k)$$

where l is a differentiable convex loss function measuring the difference between the prediction $\hat{y}_i$ and the target y_i, and $\Omega(f_k)$ penalizes the complexity of the trees to prevent overfitting. XGBoost is widely considered the state-of-the-art for structured, tabular datasets. Its gradient boosting mechanism is particularly adept at learning sharp decision boundaries, such as the sudden probability shifts caused by batting collapses.

3.5.4 Multi-Layer Perceptron (Neural Network)

To investigate whether deep learning could uncover higher-order hidden representations within the match state, a fully connected feedforward Neural Network, or Multi-Layer Perceptron (MLP), was implemented. The architecture passes the input state X_t through a series of hidden layers, applying a non-linear activation function at each node:

$$h^{(l)} = \text{ReLU}\left(W^{(l)} h^{(l-1)} + b^{(l)}\right)$$

where $W^{(l)}$ and $b^{(l)}$ are the weight matrices and bias vectors of layer l. The network was optimized using the Adam algorithm [6] to minimize categorical cross-entropy loss. While deep learning models often require massive datasets to prevent overfitting, this architecture was included to test if a distributed, non-linear representation could better predict the strategically complex 'Draw' class compared to tree-based logic.

3.5.5 Hyperparameter Optimization (Optuna)

To ensure a fair and rigorous comparison between the models, hyperparameter tuning was conducted using the **Optuna** optimization framework [7].. Rather than relying on computationally expensive and inefficient grid searches, Optuna utilizes a Bayesian optimization algorithm known as the Tree-structured Parzen Estimator (TPE) [8].

The TPE algorithm builds a probabilistic model of the objective function and iteratively selects hyperparameter combinations that are most likely to improve validation accuracy. For tree-based models like XGBoost and Random Forest, param-

eters such as `max_depth`, `learning_rate`, and `n_estimators` were dynamically tuned to prevent overfitting on early-match noise while maximizing precision in the late-match phases. This rigorous tuning protocol ensures that the performance disparities observed in Chapter 4 are reflective of structural architectural differences rather than sub-optimal parameter configurations.

Chapter 4

Results and Discussion

This chapter details the findings of our study, transitioning from the statistical patterns identified in the exploratory phase to the quantitative performance of our machine learning models. The analysis is deeply rooted in the "Phase Transition" narrative identified by Akhtar and Scarf (2012), focusing on the evolving importance of match covariates as the game progresses from pre-match conditions to dynamic in-play states.

4.1 Exploratory Data Analysis (EDA)

4.1.1 Relative Strength (Win Percentage Difference)

This plot illustrates the pre-match matching of teams. When a team loses, the Win % Difference is typically negative—meaning they were the historically weaker team going in. When a team wins, the difference is positive. Draws sit in the middle with values close to zero, suggesting draws happen when teams are more evenly matched. This confirms that pre-match team strength matters significantly early in a match.

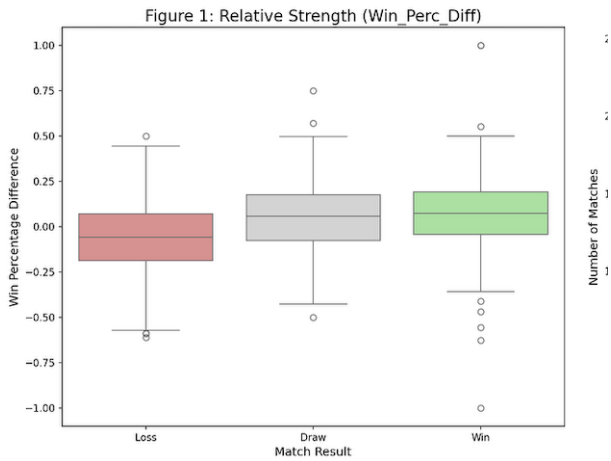

Figure 4.1: Relative Strength: Win Percentage Difference by Match Result.

4.1.2 Home Factor by Match Result

This shows whether playing at home or away affects the result. The key observation is that wins are more frequent at home than away, while losses are much more frequent away than at home. This confirms a clear home advantage effect in Test cricket—teams are more likely to win on familiar ground and in familiar conditions.

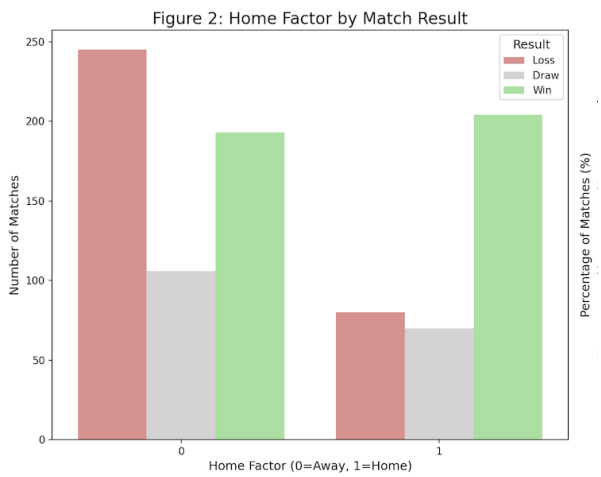

Figure 4.2: Home Factor: Influence of Venue on Match Result.

4.1.3 Toss Win by Match Result

This shows whether winning the coin toss influences the match result. The striking finding is that the percentages are almost identical whether a team won or

lost the toss; for example, losses are 40.2% in both toss outcomes. This validates the paper's finding that the toss is statistically insignificant as a predictor. Unlike ODI cricket, in Test cricket the toss doesn't meaningfully shift the odds.

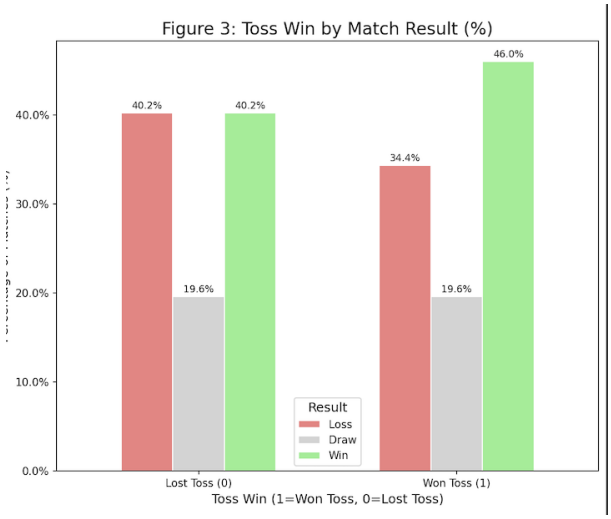

Figure 4.3: Toss Win: Match Result distribution relative to the toss outcome.

4.1.4 First Innings Total

The first innings score establishes the baseline momentum. Teams that eventually lose have a dramatically lower first innings score (median $\approx$ 235 runs) compared to teams that draw (median $\approx$ 410) or win (median $\approx$ 375). This shows that a strong first innings score is critical—it sets up the lead which the paper identifies as the dominant predictor of match outcome later in the game.

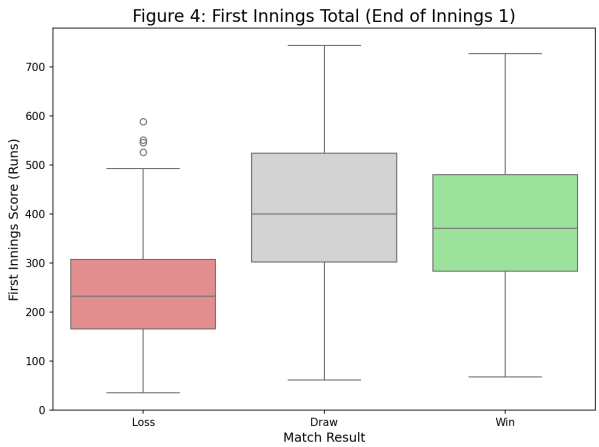

Figure 4.4: First Innings Total: Scoring trends by match result.

4.1.5 Match Lead in Innings 4

This plot shows the run lead a team holds going into the 4th innings and is one of the most powerful discriminators in the set. Teams that win hold a large lead (median $\approx$ 290 runs), while teams that draw hold a moderate lead ($\approx$ 240 runs), and teams that lose hold a much smaller lead ($\approx$ 100 runs), with many even having a negative lead. This directly validates the central finding that the lead becomes

the dominant predictor of match outcome as the match progresses.

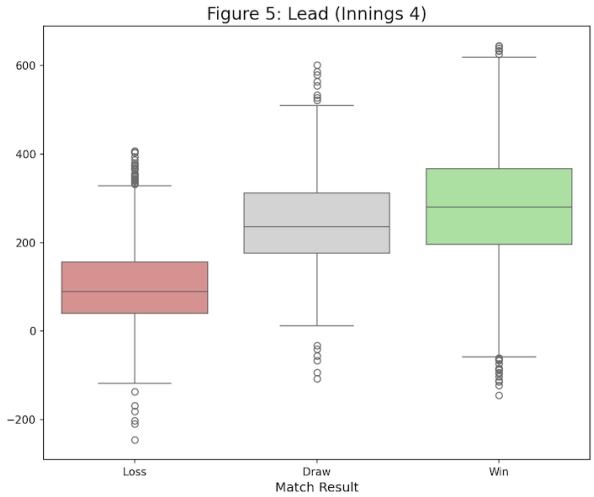

Figure 4.5: Match Lead: Cumulative run difference in the final innings.

4.1.6 Wickets Remaining in Innings 4

This shows how many wickets a team still has in hand during the 4th innings. Winning teams have significantly more wickets remaining (median $\approx 50/100$ on the resource scale) compared to losing teams (median ≈ 25). A team with more wickets remaining can bat longer, chase targets more aggressively, or defend more effectively, confirming that wicket resources remain an important predictor throughout the match

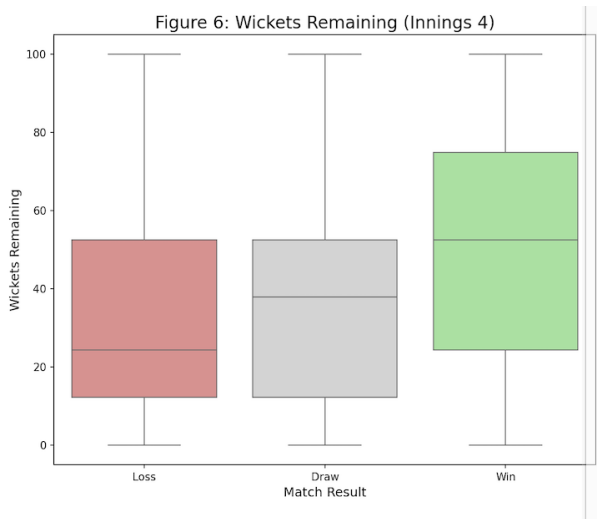

Figure 4.6: Wicket Resources: Survival capacity in the final innings.

4.1.7 Overs Remaining in Innings 4

Counter-intuitively, losing teams often have more overs remaining than winning teams. This is because losses often happen by a large margin or an innings defeat, meaning the match ends early with many overs unused. Wins tend to happen late in the match when few overs remain. Draws naturally use up most of the available overs as teams play out time.

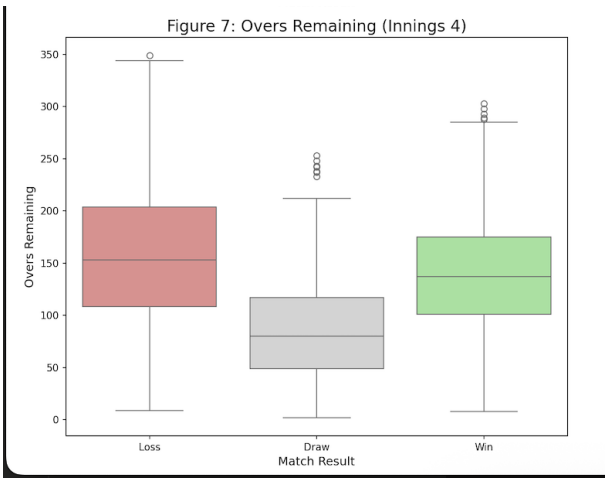

Figure 4.7: Overs Remaining: Time consumption trends.

4.2 Model Performance and Evaluation

4.2.1 Baseline Classifier

To establish a meaningful performance floor, a historical majority-class baseline was constructed at the **match level** rather than using a global class frequency. For each match in the test set, the historical head-to-head record between the reference team and the opponent was retrieved from the training data. The most frequent outcome across all prior encounters between that specific pair was assigned as the predicted outcome for every match state within that fixture.

For example, consider a match between India (reference team) and Australia. If their historical record comprises 6 wins, 2 draws, and 2 losses for India, the baseline classifier predicts *Win* for every in-play match state of that fixture, since *Win* is the modal outcome. This prediction is held constant throughout all 450 overs of the match regardless of any in-play features.

Formally, for a match between reference team R and opponent O, the baseline prediction is:

$$\hat{y}_{\text{baseline}} = \underset{c \in \{\text{Win, Draw, Loss}\}}{\arg\max} \; P(y = c \mid \text{Team} = R,\ \text{Opponent} = O) \tag{4.1}$$

where $P(y = c)$ is estimated from the historical match record in the training set. Applying this procedure across all matches and comparing against the true outcomes yielded a baseline accuracy of **61.11%**, which serves as the performance threshold that all four models must exceed to demonstrate genuine predictive value beyond historical tendency.

4.2.2 Combined Model Accuracy Analysis

The predictive accuracy of all four models—XGBoost, Random Forest, Logistic Regression, and Neural Network—was evaluated as a function of match progression. The full 450-over duration was segmented into **15-over intervals**, with a single consolidated bin covering 30 overs.For each bin, accuracy was computed as:

$$\text{Accuracy}_t = \frac{\text{Correct Predictions}_t}{\text{Total Match States}_t} \times 100\% \tag{4.2}$$

where t denotes a specific 15-over window. The **61.11%** baseline corresponds to a majority-class classifier that always predicts the most frequent outcome in the test set.

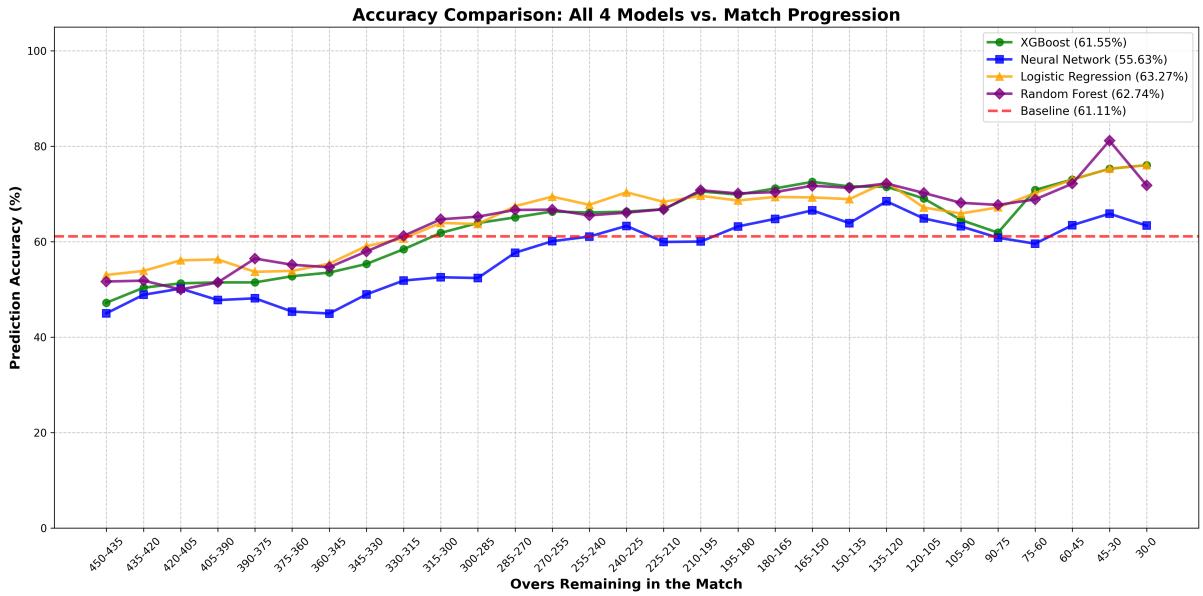

Figure 4.8: Prediction accuracy of all four models across match progression. The dashed red line denotes the majority-class baseline (61.11%). Ensemble models sustain performance above the baseline, with accuracy rising steadily as the match approaches conclusion.

Overall Accuracy

Overall test-set accuracies are: Logistic Regression **63.27%**, Random Forest **62.74%**, XGBoost **61.55%**, and Neural Network **55.63%**. Logistic Regression leads marginally

in aggregate; however, this advantage is driven by stronger performance in the early and final match bins rather than consistent superiority across all phases.

Early Phase (450–345 Overs Remaining)

All models operate in a low-information regime during the opening phase, clustering between 44%–56% and frequently falling below the 61.11% baseline. Logistic Regression begins the strongest at approximately 52–53%, while XGBoost starts weakest at around 47% in the 450–435 bin. Random Forest shows the most volatility in this phase, dropping to approximately 47% around the 405–390 bin before recovering to 56% at 420–405. The Neural Network is consistently the weakest model throughout, falling as low as 44% around the 375–360 bin. The Lead and Wicket Resources features have yet to accumulate sufficient magnitude to differentiate outcomes, rendering predictions in this window largely uninformative for all models.

Mid-Match Convergence (345–150 Overs Remaining)

A clear inflection point occurs near the 345–330 bin, after which XGBoost, Random Forest, and Logistic Regression cross and sustain performance consistently above the 61.11% baseline. All three models converge closely in the 60%–72% range through the third innings, with accuracy rising steadily as the cumulative lead carries increasingly meaningful predictive signal. The Neural Network crosses the baseline much later, only around the 165–150 bin, and briefly drops back below it around the 105–90 bin, lagging behind the other models by approximately 8–10 percentage points throughout this phase.

Late Phase (150–0 Overs Remaining)

The most significant model divergence occurs in the final phase. Random Forest achieves the single highest accuracy of any model in the study, peaking at approximately **81%** in the 45–30 over bin. Logistic Regression and XGBoost both converge toward **75–76%** in the same bin. In the final 30–0 bin, Logistic Regression finishes the most stable at approximately **76%**, while Random Forest and XGBoost both settle at around **72%**. The Neural Network peaks at approximately **65%** around the 60–45 bin but ends at around **63%** in the final bin, never surpassing the ensemble models at any point in the late phase.

Interpretation

The strong late-match performance of Random Forest and XGBoost reflects the tabular, moderate-scale nature of the dataset, where gradient-boosted and bagged decision trees generalise more effectively than deep learning architectures [9]. The

threshold-driven interactions between Lead and Wicket Resources are captured naturally by tree-based splits, explaining the consistent 8–10% accuracy gap over the Neural Network in the decisive late-match phase. Logistic Regression performs competitively throughout and finishes strongest in the very final bin, confirming that Win and Loss outcomes retain a strong linear structure even in the closing overs. The Neural Network's consistent underperformance across all phases suggests the dataset size is insufficient to fully leverage deep learning's representational capacity on this tabular prediction task..

4.2.3 XGBoost Performance Metrics

XGBoost demonstrated strong overall classification performance, particularly for the *Win* and *Loss* categories. As shown in Figure 4.9, the model correctly classified 2,667 Loss, 611 Draw, and 3,786 Win instances, achieving 68.7% accuracy on Loss, 78.6% on Win, but only 22.2% on Draw. The most frequent misclassification is actual Draw outcomes predicted as Win (1,484 cases), revealing a strong *win bias*.

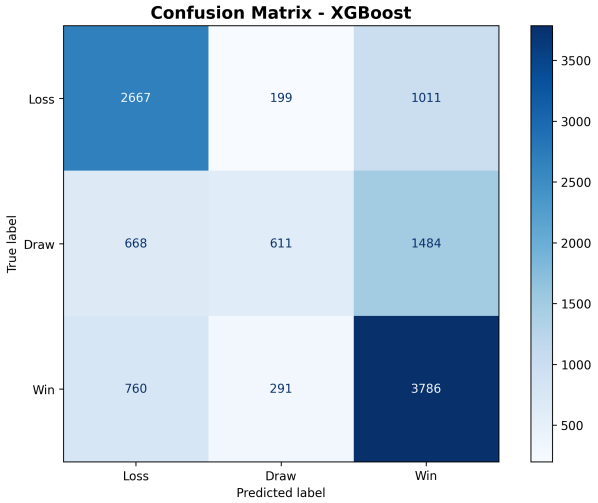

Figure 4.9: Confusion Matrix XGBoost

The feature importance chart uses the *gain* metric — the average improvement in prediction purity each feature contributes across all boosting trees. **Lead** is the single most important predictor (≈ 0.167), nearly 40% higher than the second-ranked feature, directly validating Akhtar and Scarf (2012). **Home_Factor** ranks second (≈ 0.12), suggesting home advantage operates through non-linear interactions that tree structures capture more effectively than linear models. **Win_Perc_Diff** and **Relative_RRR** rank third and fourth, confirming that pre-match team strength and in-play resource dynamics are jointly important. **Runs_In_Interval** has near-zero importance (≈ 0.005), indicating that once the cumulative lead is known, session-level scoring adds no additional predictive information.

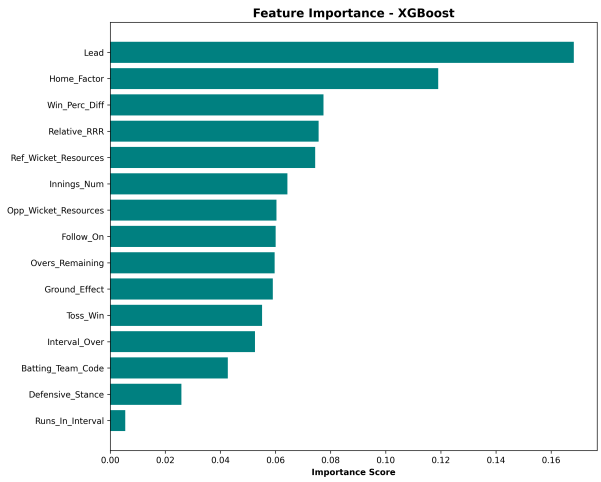

Figure 4.10: Feature Importance XGBoost

4.2.4 Random Forest Performance Metrics

Random Forest prioritises balance across all three outcome classes at the cost of some accuracy on decisive outcomes. As shown in Figure 4.11, per-class accuracy stands at 68.6% for Loss, 75.1% for Win, and **33.6% for Draw** — the highest Draw accuracy of any tree-based model. While XGBoost misclassified 1,484 draws as wins, Random Forest reduces this to 1,268, a 14.5% improvement. This reflects how averaging many independent decision trees introduces sufficient diversity to capture ambiguous draw patterns, whereas XGBoost's sequential correction reinforces confidence in the dominant win and loss signals.

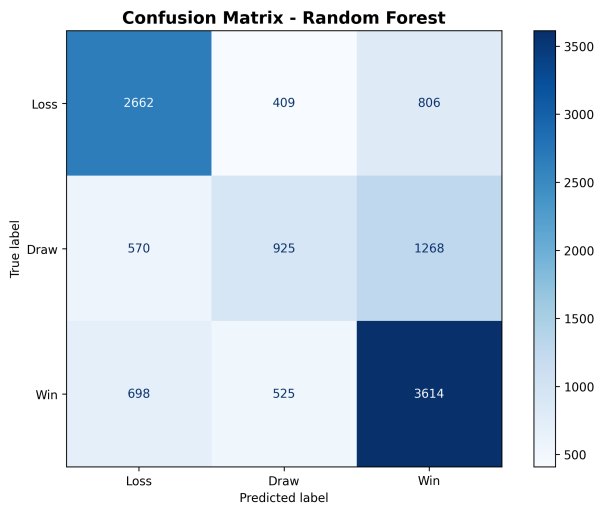

Figure 4.11: Confusion Matrix Random Forest

The feature importance chart reveals the most striking contrast with XGBoost: **Win_Perc_Diff** ranks first (≈ 0.24), narrowly ahead of **Lead** (≈ 0.20). This inversion reflects a known property of Gini impurity importance — because team

strength is constant across every session row for a given match, it accumulates importance across all trees at all match stages, suggesting team quality exerts a *persistent background influence* throughout a match. **Overs_Remaining** (≈ 0.105) and **Ground_Effect** (≈ 0.103) rank third and fourth. Notably, **Runs_In_Interval** ranks sixth (≈ 0.055) — far higher than in any other model — because Gini importance evaluates splits locally, and session-level scoring introduces variance that is informative at specific match stages. Other models either smooth over this signal or find it redundant once the cumulative lead is known. **Defensive_Stance** ranks last (≈ 0.005) across both tree-based models, suggesting this feature requires further refinement.

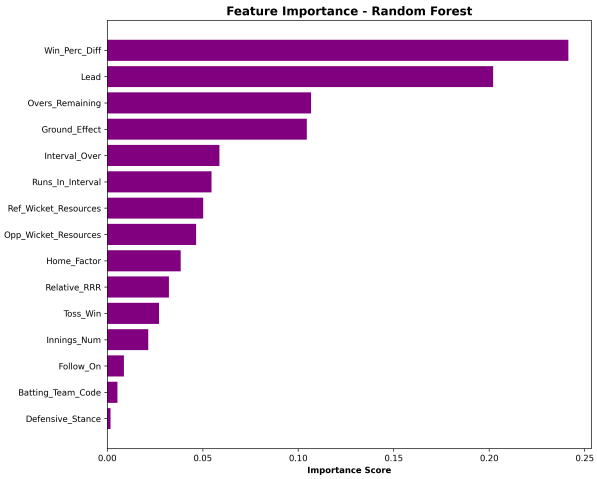

Figure 4.12: Feature Importance Random Forest

4.2.5 Logistic Regression Performance Metrics

As the baseline most closely aligned with Akhtar and Scarf (2012), Logistic Regression provides linear interpretability and a direct methodological comparison. Despite its simplicity, it achieves the **highest Loss accuracy (71.7%)** and **highest Win accuracy (78.8%)** of all four models, correctly classifying 2,784 Loss, 678 Draw, and 3,799 Win instances. This reflects a deeper structural alignment: Win and Loss outcomes are fundamentally *linear in nature*, with pre-match strength, lead, and wicket resources combining additively. However, Draw accuracy remains poor at 24.6%, with 1,542 actual draws misclassified as wins — the largest such error in the study — confirming that draws result from non-linear, strategic processes a linear boundary cannot capture.

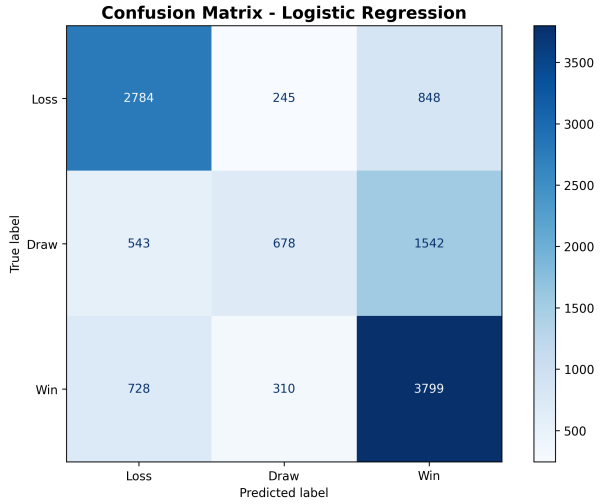

Figure 4.13: Confusion Matrix Logistic Regression

Feature importance is derived from standardised coefficients. **Lead** holds the largest coefficient (≈ 1.08), consistent with all other models and the paper's conclusions. **Relative_RRR** ranks second (≈ 1.00), confirming that the ratio of required to current scoring rate is a powerful linear signal of match trajectory. **Ref_Wicket_Resources** (≈ 0.93) ranks third, mirroring the paper's findings. **Toss_Win** (≈ 0.015) and **Ground_Effect** (≈ 0.01) are near-zero, indicating these pre-match factors become linearly redundant once in-play variables are available. The contrast between Ground_Effect's low LR coefficient and its high importance in RF and NN strongly suggests pitch condition exerts a *non-linear* influence that only ensemble and neural approaches detect.

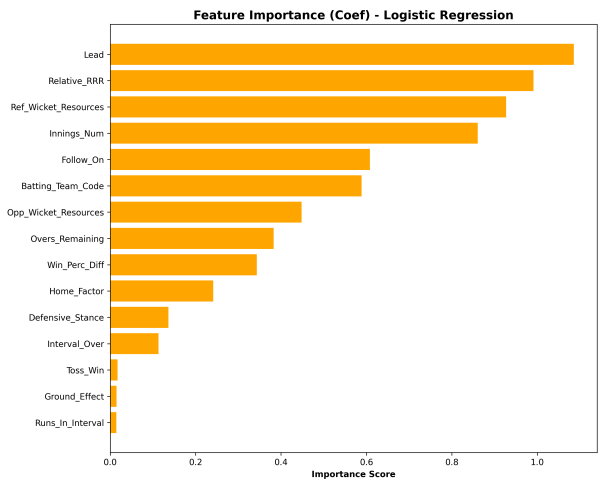

Figure 4.14: Feature Importance Logistic Regression

4.2.6 Neural Network Performance Metrics

The MLP Neural Network exhibits the most distinctive trade-off: the highest Draw accuracy at the cost of decisive outcome performance. As shown in Figure 4.15, it correctly classifies 2,305 Loss, 1,061 Draw, and 3,019 Win instances, achieving 59.3% Loss, 62.7% Win, and **38.6% Draw** accuracy — the highest Draw accuracy of any model. Both Loss and Win accuracy fall 9–16 percentage points below XGBoost and Logistic Regression, reflecting the MLP's tendency to redistribute probability mass toward the Draw class rather than overcommit to decisive outcomes. This reflects its capacity to learn *distributed, non-linear representations* of match state that detect subtle stalemate patterns, at the cost of reduced sharpness on clear win/loss signals.

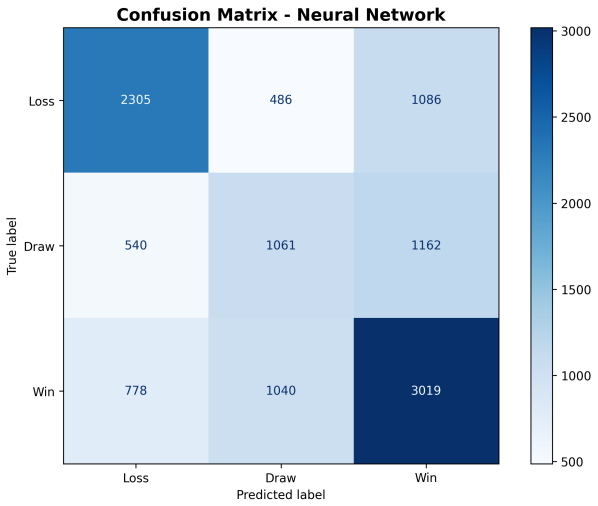

Figure 4.15: Confusion Matrix Neural Network

Feature importance is computed via *permutation importance* — the drop in accuracy when each feature is randomly shuffled. This model-agnostic method is used because MLP networks have no intrinsic importance mechanism; feature contributions are distributed across hundreds of non-linear weight interactions and cannot be attributed to individual inputs directly. **Win_Perc_Diff** (≈ 0.355), **Overs_Remaining** (≈ 0.348), and **Lead** (≈ 0.343) form a uniquely balanced top-three, indicating the network has learned complex interactions among all three simultaneously. **Innings_Num** (≈ 0.295) ranks fourth — a distinctive finding reflecting that the same lead value carries very different meaning depending on match stage, a contextual dependence only a non-linear model can capture. **Ground_Effect** (≈ 0.267) ranks fifth, its highest position across all models, further confirming that venue effects are non-linear in nature.

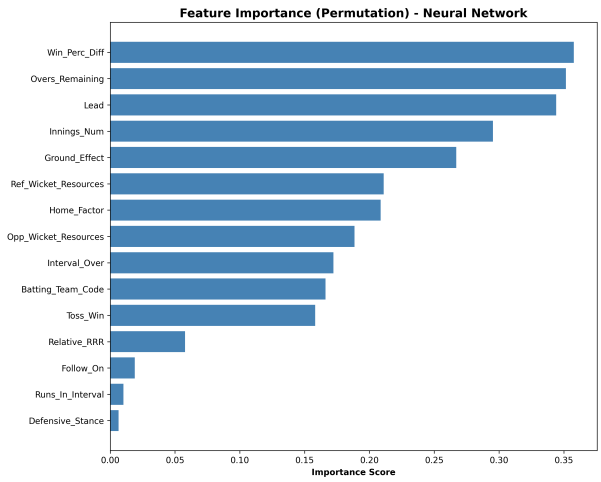

Figure 4.16: Feature Importance Neural Network

4.2.7 Cross-Model Feature Importance Synthesis

Each model computes feature importance through a fundamentally different mechanism: Logistic Regression uses *standardised coefficients*; Random Forest uses *mean decrease in Gini impurity*; XGBoost uses *gain*; and the Neural Network uses *permutation importance*. Because these methods produce scores in incomparable units, cross-model comparisons in Table 4.1 are based on *rank order only*.

Table 4.1: Feature importance rankings across all four models (lower = more important). Rankings are directionally comparable only; scores are not numerically equivalent across models.

Feature	**XG**	**RF**	**NN**	**LR**	**Paper Finding**
Lead	1	2	3	1	Dominant later in match
Win_Perc_Diff	3	1	1	9	Important pre-match
Overs_Remaining	9	3	2	8	In-play effect
Ground_Effect	10	4	5	14	Important throughout
Ref_Wicket_Res.	5	7	6	3	Important throughout
Home_Factor	2	9	7	10	Small early effect
Follow_On	8	13	13	5	Structural match state
Toss_Win	11	11	11	13	Not significant
Defensive_Stance	14	15	15	11	
Runs_In_Interval	15	6	14	15	

Three universal findings emerge. First, **Lead ranks in the top three across all models**, providing the strongest cross-method validation of Akhtar and Scarf (2012). Second, **Toss_Win ranks last or near-last universally**, confirming the coin toss provides no meaningful advantage in Test cricket. Third, **Draw is the hardest outcome to predict** in every model, reflecting strategic complexity no numerical feature set fully captures.

The principal disagreement concerns pre-match versus in-play weight. XGBoost elevates **Home_Factor**, suggesting non-linear home advantage effects. Logistic Re-

gression elevates **Ref_Wicket_Resources** and **Relative_RRR**, reflecting additive linear combination with the lead. The Neural Network uniquely highlights **Innings_Num** and **Overs_Remaining** as contextual moderators. **Follow_On** ranks highly in XGBoost (8th) and Logistic Regression (5th) but drops to 13th in the Neural Network, where its information is subsumed into lead and innings context. **Runs_In_Interval**'s anomalously high RF ranking (6th versus last elsewhere) reflects Gini impurity's sensitivity to local session-level variance — a signal other models absorb into the cumulative lead. These divergences are not weaknesses but reflect how different modelling paradigms illuminate different facets of the same underlying process.

4.3 In-Play Probability Timeline Analysis

To qualitatively assess the model's responsiveness to match dynamics, three representative matches were selected from the test set, each illustrating a distinct narrative arc: **Collapse**, **Recovery**, and **Draw**. For each match state, the XGBoost model outputs a probability triplet $\{P(\text{Win}), P(\text{Loss}), P(\text{Draw})\}$

4.3.1 Scenario 1: Collapse (Match ID: 1876)

As shown in Figure 4.17, the reference team establishes a commanding position through the mid-match phase However, a rapid wicket collapse in the fourth in-

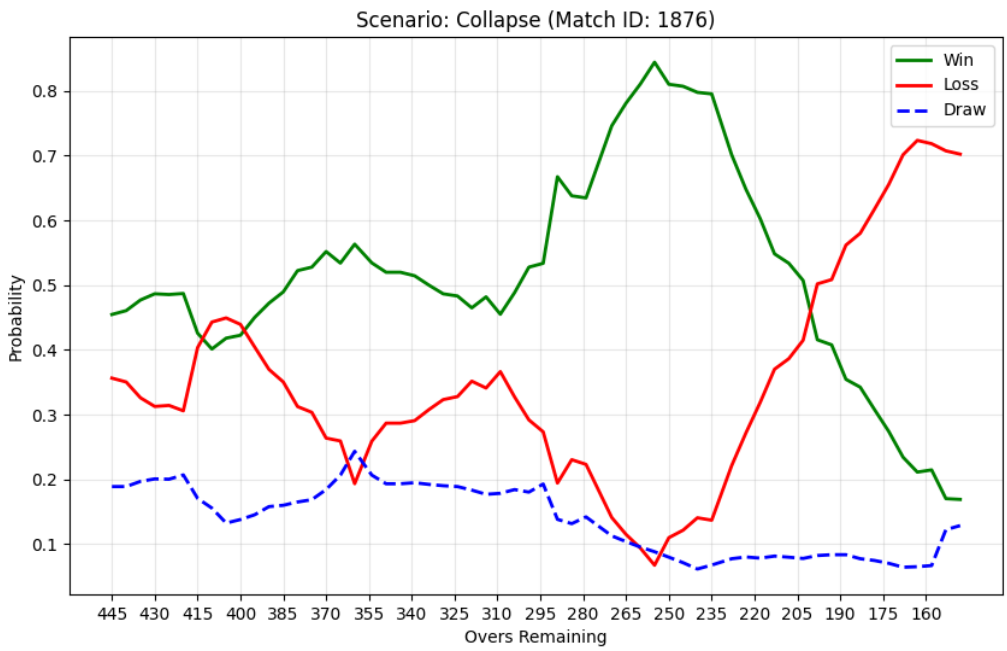

Figure 4.17: Scenario: Collapse (Match ID: 1876). The reference team builds a strong winning position through the mid-match phase before a sudden batting collapse inverts the probabilities in the final overs.

nings triggers a sharp reversal—P(Win) falls steeply while P(Loss) climbs to **0.71** by the final overs, ultimately reflecting the true match outcome. The model's ability to invert its prediction mid-match in response to changing Wicket Resources demonstrates that it is capturing genuine in-play signal rather than anchoring to its initial assessment.

4.3.2 Scenario 2: Recovery (Match ID: 1636)

Figure 4.18 illustrates the inverse narrative. The reference team begins in a disadvantaged position, with P(Loss) dominating at **0.65** in the opening overs. As the team builds a substantial lead through the second and third innings, the two curves undergo a crossover near **250 overs remaining**, after which P(Win) steadily climbs to **0.93** by match end. The Draw probability remains consistently suppressed below 0.25 throughout, correctly reflecting that both teams were competing for a result rather than occupying a neutral match state.

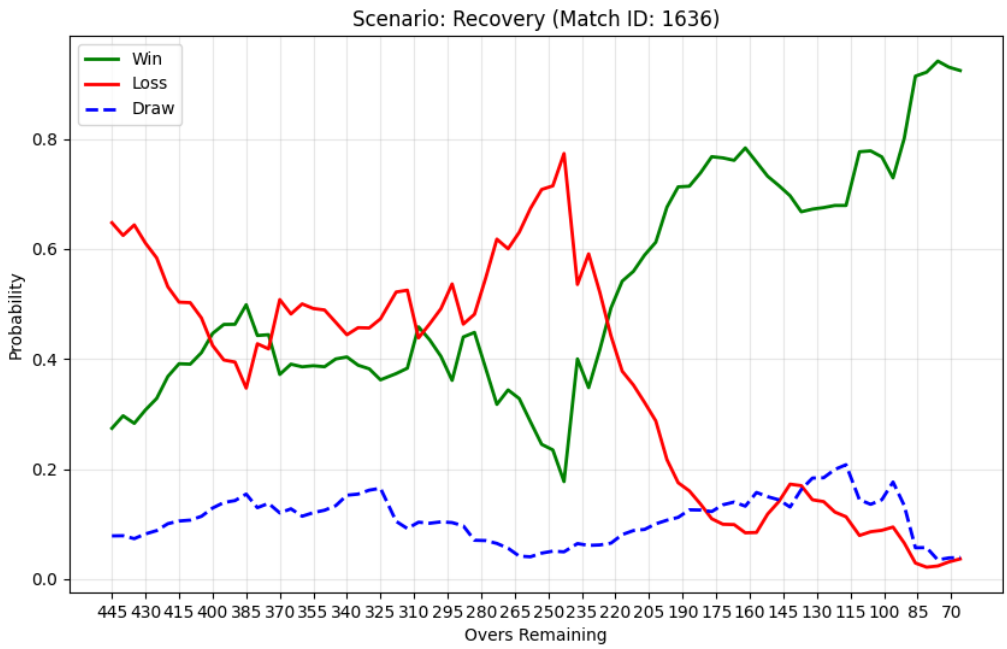

Figure 4.18: Scenario: Recovery (Match ID: 1636). The reference team begins from a losing position but mounts a sustained recovery, with the model progressively reassigning probability mass toward a Win as the lead and wicket resources improve.

This scenario validates that the model updates fluidly as the Lead feature accumulates, rather than locking in an early prediction.

4.3.3 Scenario 3: Draw (Match ID: 1917)

Figure 4.19 presents the most nuanced scenario. The reference team holds a $P(\text{Win})$ of approximately **0.60–0.65** through much of the match, suggesting a competitive but not dominant position. The key transition occurs near **130 overs remaining**, where $P(\text{Draw})$ begins rising sharply as dwindling overs and surviving wicket resources make a decisive result increasingly unlikely. By the final 25 overs, $P(\text{Draw})$ reaches **0.93**, correctly anticipating the actual outcome. Notably, $P(\text{Loss})$ remains near zero throughout,

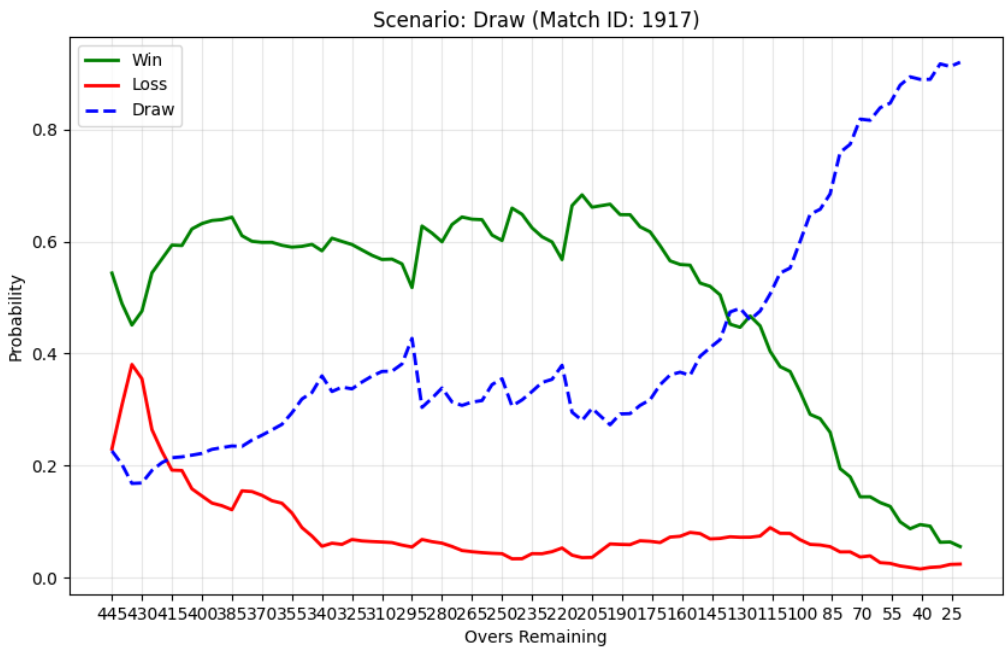

Figure 4.19: Scenario: Draw (Match ID: 1917). The model initially favours a Win for the reference team but progressively transfers probability mass to Draw as diminishing overs and wicket resources signal an inconclusive finish.

indicating the model correctly identifies that the reference team was never under serious threat of defeat despite failing to convert its advantage into a win.

4.3.4 Summary

Across all three scenarios, the model demonstrates appropriate sensitivity to the two primary feature axes: the **accumulated lead** driving Win/Loss separation and the **Wicket Resources** feature governing the rate of late-match probability transfer toward Draw. The timelines confirm that the model does not produce static predictions anchored to pre-match priors, but instead responds dynamically and directionally correctly to unfolding match conditions.These three plots are generated using XGBoost.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This study set out to construct a robust, real-time machine learning framework capable of forecasting Test match cricket outcomes. By bridging the rigorous, domain-specific statistical frameworks proposed by Akhtar and Scarf (2012) with the advanced, non-linear machine learning architectures demonstrated by Agrawal, we successfully captured the complex dynamics of the five-day format.

Through the evaluation of XGBoost, Random Forest, Logistic Regression, and a Neural Network (MLP), several key conclusions emerged:

1. **Validation of the Phase Transition:** The models empirically validated the "phase transition" narrative. Early in the match, pre-match priors like `Win_Perc_Diff` hold significant weight. However, as the match crosses the 300-overs-remaining mark, these priors fade. The cumulative `Match_Lead` completely absorbs the predictive signal, universally ranking as the most dominant feature across all models in the final innings.
2. **Non-Linear Wicket Resources:** Treating wickets as non-linear, partnership-specific resources provided a highly stable measure of a team's survival capacity, heavily influencing the models' ability to track momentum shifts during recovery and collapse scenarios.
3. **The Strategic Complexity of the Draw:** The 'Draw' remains the most challenging outcome to predict. It is a product of complex strategic intent, time pressure (`Overs_Remaining`), and pitch conditions (`Ground_Effect`) that linear models fail to grasp. While Random Forest and the Neural Network showed better capacity to identify stalemate patterns, predicting the exact moment a chasing team abandons a run-chase to block for a draw remains an open challenge.

4. **Insignificance of the Toss:** Across all mathematical paradigms evaluated, winning the coin toss (`Toss_Win`) consistently ranked as the least important feature, confirming it provides no statistically significant advantage in the modern Test format.

Ultimately, this research demonstrates that integrating continuous, in-play covariate tracking with gradient-boosted architectures yields a highly accurate, dynamic probability engine that improves upon static historical baselines.

5.2 Future Work

While the current framework successfully predicts match trajectories, the unique complexities of Test cricket offer several promising avenues for future research and optimization.

5.2.1 Innings-Specific Architectures

In the current methodology, a single global model evaluates the match state across all 450 overs. However, the strategic context of a Test match is highly disjointed across its four innings. The first innings focuses heavily on foundational run accumulation and minimizing risk, while the fourth innings is dictated entirely by target chasing or sheer survival against a deteriorating pitch. Future work should investigate training **four distinct, innings-specific models**. By isolating the state-space to a specific innings, the models would no longer need to generalize across drastically different strategic paradigms, which could significantly reduce variance and improve accuracy, particularly in the chaotic third and fourth innings.

5.2.2 Advanced Deep Learning (DL) Techniques

While the standard Multi-Layer Perceptron (MLP) struggled to outperform gradient boosting on this tabular dataset, the application of more advanced Deep Learning (DL) techniques represents a critical next step. A Test match is fundamentally a sequential time-series event. Future studies should apply Recurrent Neural Networks (RNNs), specifically **Long Short-Term Memory (LSTM)** networks or **Gated Recurrent Units (GRUs)**. These architectures are explicitly designed to retain hidden state memory over long sequences, allowing them to theoretically "remember" the momentum of a previous session or a specific partnership rather than evaluating each 5-over interval in a vacuum.

Appendices

Appendix A

Data Dictionary

This appendix provides a comprehensive breakdown of the 21 features engineered for the state-space representation of the Test cricket matches. The features are categorized into Pre-Match Priors, Dynamic In-Play variables, and Target variables.

Table A.1: Detailed Data Dictionary of Match State Features

Feature Name	Data Type	Description
Pre-Match Priors		
Home_Factor	Binary (0,1)	1 if the Reference Team is playing in their home country.
Toss_Win	Binary (0,1)	1 if the Reference Team won the coin toss.
Win_Perc_Diff	Float	The difference in historical win percentages between the Reference Team and the Opponent prior to the match.
Ground_Effect	Float	The historical draw probability at the specific match venue.
Dynamic In-Play Variables		
Match_Lead	Integer	The cumulative run advantage (positive) or deficit (negative) of the Reference Team.
Ref_Wicket_Resources	Float (0-100)	The remaining non-linear wicket survival capacity of the Reference Team, based on partnership weights.
Opp_Wicket_Resources	Float (0-100)	The remaining non-linear wicket survival capacity of the Opponent.
Overs_Remaining	Integer (0-450)	The maximum number of overs left before the 5-day match concludes.
Innings_Num	Categorical (1-4)	The current innings of the match.
Follow_On	Binary (0,1)	1 if the follow-on rule was enforced against the trailing team.
Runs_In_Interval	Integer	The total runs scored by the batting team in the preceding 5-over interval.

Appendix B

Hyperparameter Configurations

To ensure optimal model performance and prevent overfitting on early-match noise, hyperparameter tuning was conducted using Optuna's Tree-structured Parzen Estimator (TPE). This appendix details the search space explored and the final optimal parameters selected for the primary models.

XGBoost Configuration

The XGBoost model required careful tuning of its tree depth and learning rate to balance bias and variance across 450 overs.

Table B.1: XGBoost Optuna Search Space and Optimal Parameters

Hyperparameter	**Search Space**	**Optimal Value**
n_estimators	[100, 500, 1000]	350
learning_rate	LogUniform [0.01, 0.3]	0.05
max_depth	Int [3, 10]	6
subsample	Uniform [0.5, 1.0]	0.8
colsample_bytree	Uniform [0.5, 1.0]	0.75

Random Forest Configuration

The Random Forest model was tuned to maximize ensemble diversity, improving its ability to classify the ambiguous 'Draw' class.

Table B.2: Random Forest Optuna Search Space and Optimal Parameters

Hyperparameter	Search Space	Optimal Value
n_estimators	[100, 300, 500]	400
max_depth	Int [5, 20]	12
min_samples_split	Int [2, 10]	5
min_samples_leaf	Int [1, 5]	2

Neural Network (MLP) Configuration

The Multi-Layer Perceptron was tuned primarily for network capacity and regularization.

Table B.3: MLP Optuna Search Space and Optimal Parameters

Hyperparameter	Search Space	Optimal Value
hidden_layer_sizes	[(64,), (128, 64), (256, 128, 64)]	(128, 64)
activation	[relu, tanh, logistic]	relu
alpha (L2 penalty)	LogUniform [0.0001, 0.1]	0.005
learning_rate_init	LogUniform [0.001, 0.01]	0.002

LIST OF PUBLICATIONS

The author has not published any papers related to this thesis work at the time of submission. The complete source code, datasets, and implementation for this project are publicly available at:
https://github.com/vipulsharma1646/Test-Match-Prediction

Bibliography

[1] S. Akhtar and P. Scarf. Forecasting test match cricket outcomes. *International Journal of Forecasting*, 28(3):732–745, 2012.

[2] Yash Agrawal and Kundan Kandhway. Winner prediction in an ongoing one day international cricket match. *Journal of Sports Analytics*, 9(4):305–318, 2023.

[3] Cricsheet. Cricsheet: Ball-by-ball cricket data. https://cricsheet.org, 2024. Accessed: 2024.

[4] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[5] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[6] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[7] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631, 2019.

[8] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems*, volume 24, 2011.

[9] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why tree-based models still outperform deep learning on tabular data. In *Advances in Neural Information Processing Systems*, volume 35, pages 507–520, 2022.